

---

**РАБОЧИЙ ПРОТОКОЛ И ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 1 "[C++  
и UNIX]: UNIX знакомство: useradd, nano, chmod, docker, GIT, CI, CD"**

---

Группа: Z3344  
Студенты: Поздеев Дмитрий  
Преподаватель: Маслов И.

К работе допущен:  
Работа выполнена:  
Отчет принят:

---

## 1 Цель работы

- Познакомить студента с основами администрирования программных комплексов в ОС семейства UNIX,
- продемонстрировать особенности виртуализации и контейнеризации,
- продемонстрировать преимущества использования систем контроля версий (на примере GIT)

## 2 Задачи, решаемые при выполнении работы

- ОС: Работа в ОС, использование файловой системы, прав доступа, исполнение файлов
- КОНТЕЙНЕР: docker build / run / ps / images
- GitHub: / GitLab, в котором будут содержаться все выполненные ЛР
- SAVE: Всё, что было сделано в шагах 1-3, сохранить в репозиторий (+ отчет по данной ЛР в папку doc). Фиксацию ревизий производить строго через ветку dev. С помощью скриптов накатить ревизии на stg и на prd.

## 3 Реализация через Docker и VS code

### 3.1 Директории и файлы доступа

Листинг 1: Создаем директорию группы и пользователей

```
mkdir -p /usr/local/folder_max /usr/local/folder_min
groupadd group_max
groupadd group_min
useradd -m -G group_max user_max_1
useradd -m -G group_min user_min_1
```

Листинг 2: Изменяем права

```
chown -R user_max_1:group_max /usr/local/folder_max
chown -R user_max_1:group_max /usr/local/folder_min
chmod -R 770 /usr/local/folder_max
chmod -R 770 /usr/local/folder_min
chown -R user_min_1:group_min /usr/local/folder_min
chmod -R 770 /usr/local/folder_min
```

Листинг 3: Запуск скрипта

```
su - user_max_1
echo '#!/bin/bash_date >> /usr/local/folder_max/output.log' >
/usr/local/folder_max/script_max.sh
chmod +x /usr/local/folder_max/script_max.sh
/usr/local/folder_max/script_max.sh
```

### 3.2 Контейнер: docker и тд

Листинг 4: Создаем докерфайл

```
FROM ubuntu:latest
RUN apt update && apt install nano -y
COPY script.sh /app/script.sh
RUN chmod +x /app/script.sh
CMD ["/app/script.sh"]
```

Листинг 5: Создаем скрипт, собираем образ и запускаем контейнер

```
#!/bin/bash
date >> /app/output.log
docker build -t my_image .
docker run -it --rm my_image
docker run --rm my_image getent passwd
```

### 3.3 Работа с Git

Листинг 6: Клонировем репозеторий и создаем структуру

```
git clone https://github.com/PozdeevDA/my_labs.git
```

```

cd my_labs
mkdir -p lab_01/{build,src,doc,cmake} lab_02/{build,src,doc,cmake}
git checkout -b dev
git push -u origin dev
git branch -d main
git push origin --delete main

```

Листинг 7: Скрипт для переноса из dev в stg:

```

#!/bin/bash
git checkout stg
git merge dev
git tag "stg-$(date +%Y%m%d%H%M)"
git push origin stg --tags

```

### 3.4 Сохранение результатов

Листинг 8: Скрипт для переноса из dev в stg:

```

git add .
git commit -m "last_fix"
git push origin dev

```

## 4 Выводы

В ходе работы я освоил основы администрирования UNIX: создание пользователей, групп, настройку прав доступа (chmod, chown) и написание скриптов, а также убедился в важности изоляции окружения через Docker (сборка образов, запуск контейнеров). Организовал Git-репозиторий с ветками dev/stg/prd, автоматизировал перенос изменений скриптами, что имитирует CI/CD-процессы. Столкнулся с ограничениями Windows, но решил их через WSL и Docker, закрепив навыки работы в UNIX-средах и управления версиями.